

***“Aplicación de estructuras jerárquicas, grafos
y sus algoritmos asociados”***

Integrantes del grupo: Leandro Valentín Mendieta, Alex Luciano Pedrón.

Fecha de entrega: 08/06/2026

Link de repositorio: <https://github.com/Valentin074/AyED2026c1---Mendieta-Pedron.git>

Introducción

El presente trabajo práctico tiene como objetivo aplicar y analizar distintas estructuras de datos abstractas. A través de tres problemas diferentes se busca evaluar la capacidad de seleccionar, implementar y utilizar estructuras adecuadas según los requerimientos de cada situación planteada.

En la primera actividad se modela el funcionamiento de una sala de emergencias hospitalaria, donde los pacientes deben ser atendidos según su nivel de prioridad médica. Para resolver este problema se emplea una Cola de Prioridad, permitiendo administrar eficientemente el orden de atención.

La segunda actividad consiste en el desarrollo de una base de datos de temperaturas históricas. Para almacenar y consultar las mediciones se utiliza un Árbol AVL, estructura que garantiza búsquedas, inserciones y eliminaciones eficientes gracias a su balanceo automático.

Finalmente, la tercera actividad aborda un problema de distribución de noticias entre aldeas conectadas mediante una red de caminos. Para representar las conexiones se utiliza un grafo pesado. También con el objetivo de minimizar la distancia total recorrida durante la transmisión de información.

A lo largo del informe se analizan las decisiones de diseño adoptadas, la implementación realizada y los resultados obtenidos para cada uno de los problemas propuestos.

Desarrollo de los problemas

Problema N°1: Sala de Emergencias Hospitalaria:

A. Análisis y planteo

El problema consiste en simular el funcionamiento de una sala de emergencias donde los pacientes ingresan continuamente y deben ser atendidos según la gravedad de su estado de salud. La operación más importante es determinar qué paciente debe ser atendido primero. Debido a que los pacientes poseen distintos niveles de riesgo, una cola convencional no resulta adecuada, ya que atiende estrictamente por orden de llegada.

Para resolver esta situación se seleccionó una Cola de Prioridad. Esta estructura permite asociar una prioridad a cada elemento almacenado y garantiza que al desencolar se obtenga siempre el elemento con mayor prioridad.

Los niveles de riesgo utilizados son:

- **Riesgo 1:** Crítico.
- **Riesgo 2:** Moderado.
- **Riesgo 3:** Bajo.

Además, para mantener el orden de llegada entre pacientes con igual riesgo, se utiliza un contador de ingreso como segundo criterio de prioridad. De esta manera se garantiza un comportamiento justo para pacientes con la misma gravedad. La elección de una Cola de Prioridad resulta adecuada debido a que las operaciones predominantes son inserción de nuevos pacientes y extracción del paciente más urgente, ambas realizadas de manera eficiente.

B. Implementación / Resolución

La solución se implementó mediante la clase ColaPrioridad. Cada vez que ingresa un paciente se genera un objeto de la clase Paciente, el cual contiene su nombre, apellido, nivel numérico de riesgo y descripción textual.

La prioridad utilizada para el almacenamiento interno en el Montículo se construye mediante una tupla ordenada:

Prioridad = (prioridad_medica, orden_de_llegada)

Durante la simulación de ejecución continua, ingresan un total de 20 pacientes distribuidos de forma aleatoria en el tiempo. Luego de cada ingreso, se simula mediante probabilidad (50%) si un médico se libera para atender. El sistema expone por consola en tiempo real las altas en la sala de espera y las llamadas a los consultorios, aplicando estrictamente la lógica de la tupla de prioridad.

C. Resultados y pruebas

- Los pacientes de **Riesgo 1** pasaron directamente al frente de la cola de atención sin importar cuántos pacientes de Riesgo 2 o 3 hubieran ingresado previamente.
- Ante empates de riesgo médico, el contador secuencial resolvió de manera equitativa atendiendo en orden de llegada cronológico.
- Al finalizar el ingreso de los 20 pacientes de prueba, la simulación procesó el vaciado ordenado de la sala hasta que el contador de pendientes llegó a cero.

Adjuntamos prueba cuando ingresa un paciente de nivel 1 (critico) y el proceso hasta que es atendido.

05/06/2026 14:51:57

Ingresa a sala de espera: Mariela García -> 1-crítico
Los médicos se encuentran ocupados en el quirófano...

Pacientes que faltan atenderse: 5

Mariela García	-> 1-crítico
Estela Juarez	-> 2-moderado
Leandro Colman	-> 3-bajo
Agustina Juarez	-> 3-bajo
Andrea Rodriguez	-> 2-moderado

05/06/2026 14:51:58

Ingresa a sala de espera: Gastón García -> 2-moderado
Los médicos se encuentran ocupados en el quirófano...

Pacientes que faltan atenderse: 6

Mariela García	-> 1-crítico
Estela Juarez	-> 2-moderado
Gastón García	-> 2-moderado
Agustina Juarez	-> 3-bajo
Andrea Rodriguez	-> 2-moderado
Leandro Colman	-> 3-bajo

05/06/2026 14:51:59

Ingresa a sala de espera: Estela Rodriguez -> 2-moderado

Se atiende el paciente: Mariela García -> 1-crítico

Pacientes que faltan atenderse: 6

Estela Juarez	-> 2-moderado
Andrea Rodriguez	-> 2-moderado
Gastón García	-> 2-moderado
Agustina Juarez	-> 3-bajo
Estela Rodriguez	-> 2-moderado
Leandro Colman	-> 3-bajo

La lista de pacientes en espera refleja la disposición interna del montículo binario (que no representa un ordenamiento lineal total), pero se demuestra que el algoritmo de desencolado extrae estrictamente el mínimo de acuerdo con las prioridades.

Problema N°2: Base de Datos de Temperaturas

A. Análisis y planteo

El problema consiste en almacenar registros de temperatura asociados a fechas específicas y permitir consultas eficientes sobre dichos datos. Las operaciones requeridas son inserción, búsqueda por fecha, eliminación y consultas analíticas por rangos de fechas (máximos y mínimos).

Debido a la necesidad de mantener los registros ordenados cronológicamente y realizar búsquedas rápidas, se seleccionó un Árbol AVL. Los árboles AVL son árboles binarios de búsqueda auto-balanceados que garantizan una complejidad de tiempo $O(\log n)$ para las operaciones principales. Esta característica estructural es la que previene que el árbol se degrade en una lista enlazada si los datos entran ordenados por fecha de forma secuencial.

B. Implementación / Resolución

La clase Temperaturas_DB encapsula la lógica del Árbol AVL donde la clave es la fecha y el valor asociado es la temperatura.

El método clave implementado para cumplir la consigna analítica es `obtener_en_rango(fecha_inicio, fecha_fin)`. Este método realiza un recorrido recursivo inteligente: evalúa el valor de la clave del nodo actual y decide si debe podar ramas completas. Si la fecha del nodo es menor que `fecha_inicio`, se descarta por completo su subárbol izquierdo, logrando optimizar drásticamente la velocidad de respuesta. A partir de la lista filtrada, los métodos de negocio calculan los picos del rango.

C. Resultados y pruebas

Al ejecutar el sistema con el set de datos provisto, se obtuvieron las siguientes métricas y respuestas por consola:

- **Carga Masiva:** Se procesó con éxito el archivo físico `muestras.txt`, insertando y balanceando un total de 120 muestras de temperatura en la estructura AVL.
- **Consulta Puntual:** La búsqueda de la fecha "15/01/2025" localizó de forma inmediata la temperatura registrada de 36.9 °C.
- **Búsqueda por Rangos:** Se solicitó el análisis del rango comprendido entre el "05/01/2025" y el "20/01/2025". El sistema extrajo el subconjunto ordenándolo cronológicamente de manera nativa e informó con precisión matemática los extremos climáticos (Temperatura Máxima en Rango: 39.5 °C y Temperatura Mínima en Rango: 14.4 °C).
- **Eliminación y Rebalanceo:** Se ejecutó la baja del registro "10/01/2025". El árbol recalculó sus factores de balance, aplicó las rotaciones correspondientes y el sistema confirmó la actualización reduciendo la persistencia total a **119 registros**.

```
=====
BASE DE DATOS DE TEMPERATURAS - KEVIN KELVIN (PROBLEMA 2)
=====
```

```
[+] Leyendo archivo real desde: ./data/muestras.txt
-> Éxito. Total de muestras reales cargadas en el AVL: 120

[?] Operación devolver_temperatura('15/01/2025'):
-> Resultado: 36.9 °C

[?] Consultas por rangos entre [05/01/2025] y [20/01/2025]:
-> max_temp_rango: 39.5 °C
-> min_temp_rango: 14.4 °C
-> temp_extremos_rango: (14.4, 39.5)
```

```
[?] devolver_temperaturas('05/01/2025', '20/01/2025') [Listado Ordenado]:
05/01/2025: 18.6 °C
06/01/2025: 35.2 °C
07/01/2025: 31.0 °C
08/01/2025: 20.3 °C
09/01/2025: 16.0 °C
10/01/2025: 21.1 °C
11/01/2025: 39.5 °C
12/01/2025: 24.8 °C
13/01/2025: 33.6 °C
14/01/2025: 34.6 °C
15/01/2025: 36.9 °C
16/01/2025: 27.5 °C
17/01/2025: 15.3 °C
18/01/2025: 32.8 °C
19/01/2025: 14.4 °C
20/01/2025: 19.4 °C

[-] Borrando la temperatura de la fecha: '10/01/2025'
-> Nueva cantidad de muestras totales en la BD: 119

=====
```

D. Análisis de Complejidad Temporal (Big-O)

A continuación, se detalla la complejidad en el peor caso para cada método de Temperaturas_DB:

Operación / Método	Complejidad (Peor Caso)	Justificación
guardar_temperatura()	$O(\log n)$	Búsqueda binaria de la posición e inserción. Las rotaciones de balanceo toman tiempo constante $O(1)$.
devolver_temperatura()	$O(\log n)$	Búsqueda puntual descendiendo por la altura del árbol, descartando la mitad de los nodos en cada paso.
borrar_temperatura()	$O(\log n)$	Localización del nodo, búsqueda del mínimo sucesor (si aplica) y rebalanceo en el camino de regreso.
cantidad_muestras()	$O(1)$	Acceso inmediato mediante un atributo contador (self._tamano) actualizado dinámicamente.
devolver_temperaturas()	$O(\log n + k)$	Búsqueda eficiente por rango: Se aplica poda de ramas para localizar el intervalo en $O(\log n)$ y procesar solo los k elementos que caen dentro de él.
max_temp_rango / min_temp_rango	$O(\log n + k)$	Extrae los k elementos del rango usando la subrutina optimizada y busca el extremo de forma lineal sobre ese subconjunto.
cargar_desde_archivo()	$O(m \cdot \log n)$	Siendo m la cantidad de líneas del archivo. Realiza m inserciones sucesivas de costo logarítmico.

Donde n es el total de registros en el sistema, k es la cantidad de elementos dentro del rango consultado, y m es el número de líneas leídas del archivo.

Problema N°3: Sistema de Distribución de Noticias

A. Análisis y planteo

El objetivo del problema es diseñar una estrategia eficiente para distribuir noticias entre un conjunto de aldeas conectadas mediante caminos de diferentes longitudes. Las operaciones fundamentales consisten en representar la red, calcular las interconexiones sin ciclos y minimizar las distancias de vuelo de las palomas.

Para modelar la red se utilizó un Grafo Pesado, donde cada vértice representa una aldea y cada arista representa un camino con una distancia asociada en leguas. La solución matemática óptima se basa en construir un Árbol de Expansión Mínima (MST), dado que esta estructura asegura conectar la totalidad de los vértices sin bucles y minimizando estrictamente el peso total acumulado de las aristas escogidas.

B. Implementación / Resolución

La estructura se definió en la clase GrafoPesado. El cálculo de la distribución se diseñó en el módulo AgenciaNoticias.py a través de la función calcular_transmision_optima().

El algoritmo inicia fijando un nodo emisor primario ("Peligros"). En cada paso, evalúa el conjunto de fronteras de los nodos ya visitados, detecta la arista con el menor peso numérico (leguas) que apunte a una aldea no alcanzada y la añade al árbol de transmisión, repitiendo el ciclo hasta cubrir el total de los vértices del grafo.

C. Resultados y pruebas

La prueba de ejecución conectó el mapa completo de 22 aldeas:

=====		
SISTEMA DE DISTRIBUCIÓN DE NOTICIAS EFICIENTE - PALOMAS WILLIAM		
=====		
Emisor inicial del mensaje: Peligros		
ALDEA	RECIBE DE	REPLICA A (ENVIARA)

Aceituna	Malcocinado	Ninguna (Fin de rama)
Buenas Noches	La Aparecida	Cebolla
Cebolla	Buenas Noches	Pancrudo
Consuegra	Malcocinado	Ninguna (Fin de rama)
Diosleguarde	Malcocinado	Elciego
El Cerrillo	Lomaseca	Malcocinado
Elciego	Diosleguarde	Melón
Espera	La Pera	Ninguna (Fin de rama)
Hortijos	Humilladero	Ninguna (Fin de rama)
Humilladero	Torralta	Hortijos
La Aparecida	Peligros	Buenas Noches, Silla
La Pera	Los Infernos	Espera
Lomaseca	Peligros	Los Infernos, Pepino, El Cerrillo
Los Infernos	Lomaseca	La Pera
Malcocinado	El Cerrillo	Consuegra, Aceituna, Diosleguarde
Melón	Elciego	Ninguna (Fin de rama)
Pancrudo	Cebolla	Ninguna (Fin de rama)
Peligros	EMISOR ORIGEN	La Aparecida, Lomaseca
Pepino	Lomaseca	Ninguna (Fin de rama)
Silla	La Aparecida	Torralta
Torralta	Silla	Villaviciosa, Humilladero
Villaviciosa	Torralta	Ninguna (Fin de rama)

Suma total de distancias recorridas por todas las palomas: 94 leguas.		
=====		

El algoritmo garantizó la conectividad de toda la población civil minimizando los trayectos. La suma total de distancias recorridas por todas las palomas del sistema se redujo a exactamente **94 leguas**.

Análisis de resultados específicos / Discusión

Los tres problemas abordados demuestran cómo la elección adecuada de una estructura de datos impacta directamente en la eficiencia de la solución.

En el problema de la sala de emergencias, la Cola de Prioridad permitió gestionar correctamente la atención de pacientes según la gravedad de su estado, garantizando una respuesta rápida ante situaciones críticas. En la base de datos de temperaturas, el Árbol AVL ofreció tiempos de búsqueda, inserción y eliminación eficientes, además de facilitar las consultas sobre rangos de fechas gracias a la organización ordenada de los datos. Por último, el uso de grafos permitió resolver un problema de optimización de recorridos, minimizando la distancia total necesaria para transmitir información a todas las aldeas.

Declaración de Uso Ético de Inteligencia Artificial

Herramientas utilizadas:

- Gemini

Descripción del uso:

La herramienta se utilizó para plantear la estructura inicial del código y para buscar errores de lógica o sintaxis que no veíamos a primera vista durante el desarrollo. Asimismo, nos ayudó en la redacción final de este informe y sirvió como apoyo para entender algunos conceptos teóricos.

Declaración de responsabilidad:

“Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y el código presentado en este informe. Comprendemos los conceptos utilizados y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final refleja nuestro propio análisis, interpretación y comprensión de los temas abordados.”

Trabajo en equipo

Para este trabajo nos organizamos hablando por WhatsApp y armando un documento compartido en Google Docs para escribir el informe. El desarrollo se dividió entre ambos, Leandro Valentin Mendieta se encargó de plantear los códigos del problema 1 (Hospital) y el problema 3 (Sistema de Noticias), además de la organización de los codigos en el área de trabajo, mientras que Alex Luciano Pedrón se ocupó de encarar el problema 2 (Temperaturas) y el armado del informe. Hubo intenciones constantes de avanzar y predisposición de ambas partes para colaborar en los módulos.

Al principio, calculamos que el proyecto nos iba a demandar unas 25 horas de trabajo en total para cumplir con todos los requerimientos de la consigna. Luego de finalizar todas las

etapas, el tiempo real de dedicación terminó siendo de 20 horas (aproximadamente), logrando cumplir con los objetivos planteados dentro de un tiempo que consideramos razonable.

Conclusiones

La realización de este trabajo práctico permitió analizar el impacto que tiene la selección de una estructura de datos sobre la resolución de problemas específicos. Consideramos que los objetivos planteados inicialmente se han cumplido, ya que logramos poner en funcionamiento cada uno de los sistemas solicitados.

El desarrollo de los ejercicios nos demostró que la correcta elección de herramientas como colas de prioridad, árboles AVL y grafos es clave para procesar la información de forma ordenada. El proceso nos dejó como principal conclusión que el diseño de un software eficiente depende siempre de analizar bien el problema antes de programar, buscando optimizar los recursos disponibles en cada situación.

Bibliografía

- **Miller, B. N., & Ranum, D. L.** *Problem Solving with Algorithms and Data Structures using Python*. Runestone Academy. Disponible en: <https://runestone.academy/ns/books/published/pythoned/index.html>
- **Apuntes de la Cátedra.** Material teórico-práctico de la asignatura sobre Estructuras de Datos.

